```
!pip install deepdiff hdbscan xgboost shap -q
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 91.4/91.4 kB 2.3 MB/s eta 0:00:00
```

```
import os
import re
import random
import copy
import xml.etree.ElementTree as ET
import pandas as pd
import numpy as np
import shap
from deepdiff import DeepDiff
from xgboost import XGBClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import classification_report
import hdbscan
```

```
/usr/local/lib/python3.12/dist-packages/hdbscan/robust_single_linkage_.py:175: SyntaxWarning: invalid escape sequence '\{'
  $max \{ core_k(a), core_k(b), 1/\alpha d(a,b) \}$.
```

Stage 1 : XML -> Structured JSON

```
def parse_bill(xml_path):
    tree = ET.parse(xml_path)
    root = tree.getroot()

    def get_text(path, default="0"):
        elem = root.find(path)
        return elem.text if elem is not None else default

    return {
        "invoice_number": get_text("Customer/invoice_number"),
        "rate_plan": get_text("Electric/rate_plan"),
        "usage_kwh": float(get_text("Electric/usage_kwh", 0)),
        "other_charges": float(get_text("OtherCharges", 0)),
        "total_amount": float(get_text("TotalAmountDue", 0))
    }

#ensure directory exists
os.makedirs("/content/content", exist_ok=True)

xml_files = [f for f in os.listdir("/content/content/training") if f.endswith(".xml")]

if len(xml_files) == 0:
    raise Exception("No files found. Upload bills first.")

bills = [parse_bill(f"/content/content/training/{file}") for file in xml_files]

print(f"Loaded {len(bills)} XML bills.")
```

```
Loaded 13 XML bills.
```

Stage 2 : Deterministic Smart Diff

```
def inject_error(bill):
    modified = copy.deepcopy(bill)
    error_type = random.choice(["rate_error", "missing_charge", "rounding"])

    if error_type == "rate_error":
        modified["total_amount"] += random.uniform(20, 80)

    elif error_type == "missing_charge":
        deduction = random.uniform(10, 30)
        modified["other_charges"] -= deduction
        modified["total_amount"] -= deduction

    elif error_type == "rounding":
        modified["total_amount"] += random.uniform(-0.01, 0.01)

    return modified, error_type

delta_records = []

#generate synthetic old/new pairs for training
for bill in bills:
    for   in range(20):   #increase to scale dataset
```

```
        modified, error_type = inject_error(bill)

        diff = DeepDiff(bill, modified, ignore_order=True)

        delta_records.append({
            "invoice_number": bill["invoice_number"],
            "delta_total": modified["total_amount"] - bill["total_amount"],
            "delta_other": modified["other_charges"] - bill["other_charges"],
            "usage_kwh": bill["usage_kwh"],
            "error_type": error_type,
            "diff_object": str(diff)
        })

delta_df = pd.DataFrame(delta_records)

print(f"Generated {len(delta_df)} delta records.")

#map to required classification labels
def map_label(error_type):
    if error_type == "rounding":
        return "Benign"
    elif error_type in ["rate_error", "missing_charge"]:
        return "Critical_Error"
    else:
        return "Investigate"

delta_df["class"] = delta_df["error_type"].apply(map_label)
```

```
Generated 260 delta records.
```

Stage 3 : AI Classification Engine

```
features = ["delta_total", "delta_other", "usage_kwh"]

X = delta_df[features]
y = delta_df["class"]

le = LabelEncoder()
y_encoded = le.fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded, test_size=0.3, random_state=42
)

model = XGBClassifier(eval_metric='mlogloss')
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("\nclassification report:")
print(classification_report(y_test, y_pred, target_names=le.classes_))

#predict class probabilities
probs = model.predict_proba(X_test)

#predicted class index
preds = model.predict(X_test)

#convert to class labels
predicted_labels = le.inverse_transform(preds)

#get confidence (highest probability)
confidence_scores = probs.max(axis=1)


results_df = pd.DataFrame({
    "Predicted_Class": predicted_labels,
    "Confidence_%": (confidence_scores * 100).round(2)
})

print("\nsample predictions with confidence:")
print(results_df.head())
```

```
classification report:
               precision    recall  f1-score   support

       Benign       0.96      1.00      0.98        27
Critical_Error       1.00      0.98      0.99        51
```

```
          accuracy                           0.99        78
         macro avg       0.98      0.99      0.99        78
      weighted avg       0.99      0.99      0.99        78


sample predictions with confidence:
  Predicted_Class  Confidence_%
0          Benign     98.949997
1  Critical_Error     98.980003
2  Critical_Error     98.980003
3  Critical_Error     98.980003
4  Critical_Error     99.239998
```

Stage 4 : Cluster Critical Errors

```
critical_df = delta_df[delta_df["class"] == "Critical_Error"].copy()

if len(critical_df) > 1:
    cluster_features = critical_df[["delta_total", "delta_other"]]

    clusterer = hdbscan.HDBSCAN(min_cluster_size=5)
    critical_df["cluster"] = clusterer.fit_predict(cluster_features)

    print("\nCritical Error Clusters:")
    print(critical_df[["invoice_number", "delta_total", "delta_other", "cluster"]].head(20))

else:
    print("\ninsufficient critical_error samples for clustering.")

print("\npipeline completed")
```

```
Critical Error Clusters:
   invoice_number  delta_total  delta_other  cluster
0    601009895585   -23.421918   -23.421918        0
1    601009895585   -27.753397   -27.753397        0
2    601009895585    75.094145     0.000000        1
3    601009895585   -29.281240   -29.281240        0
4    601009895585   -18.368980   -18.368980        0
5    601009895585   -16.483457   -16.483457        0
6    601009895585    58.917360     0.000000        2
7    601009895585   -29.442864   -29.442864        0
8    601009895585   -19.304834   -19.304834        0
10   601009895585    32.345360     0.000000        4
12   601009895585    23.776835     0.000000        6
13   601009895585   -15.503841   -15.503841        0
15   601009895585   -26.069670   -26.069670        0
16   601009895585   -25.397067   -25.397067        0
17   601009895585   -18.933571   -18.933571        0
18   601009895585    30.288822     0.000000        4
20   604008522904   -28.430787   -28.430787        0
22   604008522904   -25.999502   -25.999502        0
26   604008522904   -28.043142   -28.043142        0
27   604008522904    60.632491     0.000000        2

pipeline completed
```

Stage 5 : Parsing of Test Data Set

```
def parse_word_xml_bill(xml_path):
    tree = ET.parse(xml_path)
    root = tree.getroot()

    namespace = {'w': 'http://schemas.microsoft.com/office/word/2003/wordml'}

    #extract all text content from Word XML
    texts = []
    for node in root.findall('.//w:t', namespace):
        if node.text:
            texts.append(node.text)

    full_text = " ".join(texts)

    #debugger: print extracted text
    print("\nExtracted text preview:")
    print(full_text)

    #regex extraction
    invoice_number_match = re.search(r'(?:Invoice|Account)\s*(?:Number|No\.)?\s*[:-]?\s*([A-Za-z0-9-]+)', full_text, re.IGN
    rate_plan_match = re.search(r'Rate\s*Plan:\s*([A-Za-z0-9\s()]+)', full_text, re.IGNORECASE)

    usage_kwh_match = re.search(r'Usage.*?(\d+\.?\d*)', full_text, re.IGNORECASE)
    total_amount_match = re.search(r'Total.*?Amount.*?(\d+\.?\d*)', full_text, re.IGNORECASE)
```

```
    other_charges_match = re.search(r'Other.*?Charges.*?(\d+\.?\d*)', full_text, re.IGNORECASE)

    return {
        "invoice_number": invoice_number_match.group(1) if invoice_number_match else "unknown",
        "rate_plan": rate_plan_match.group(1).strip() if rate_plan_match else "unknown",
        "usage_kwh": float(usage_kwh_match.group(1)) if usage_kwh_match else 0.0,
        "other_charges": float(other_charges_match.group(1)) if other_charges_match else 0.0,
        "total_amount": float(total_amount_match.group(1)) if total_amount_match else 0.0
    }
```

Stage 6 : Inference + Field-Level Error Diagnosis

```
#per-customer history seeded from training bills
customer_history = {}
for b in bills:
    customer_history.setdefault(b["invoice_number"], []).append(b)

def get_customer_baseline(invoice_number):
    #return per-customer mean/std, falling back to global average for new accounts

    hist = customer_history.get(invoice_number, [])
    src  = hist if hist else bills
    label = f"customer_history ({len(hist)} bills)" if hist else "global_fallback"
    return {
        "total_amount":  np.mean([h["total_amount"]  for h in src]),
        "other_charges": np.mean([h["other_charges"] for h in src]),
        "usage_kwh":     np.mean([h["usage_kwh"]     for h in src]),
        "total_std":     np.std([h["total_amount"]   for h in src]) or 1.0,
        "other_std":     np.std([h["other_charges"]  for h in src]) or 1.0,
        "usage_std":     np.std([h["usage_kwh"]      for h in src]) or 1.0,
        "source":        label,
    }

#SHAP explainer (built once, reused per bill)
explainer = shap.TreeExplainer(model)

def get_shap_contributions(shap_values, pred_label):
    """
    safely extract per-feature SHAP contributions for the predicted class.

    XGBoost + TreeExplainer can return shap_values in 2 shapes depending
    on the no. of classes actually present in the data:
      - binary (2 classes active): ndarray of shape (1, n_features)
      - multi-class (3+ classes):  list of ndarrays, one per class, each (1, n_features)

    normalise both cases to plain dict {feature_name: shap_value}.
    """
    feat_names = ["delta_total", "delta_other", "usage_kwh"]


    #shap.treeexplainer returning different shapes
    if isinstance(shap_values, list):
        class_idx = list(le.classes_).index(pred_label)
        # Guard: if the list is shorter than expected (binary fallback), use index 0
        idx = class_idx if class_idx < len(shap_values) else 0
        row = shap_values[idx][0]

    # Binary / single-output path: shap_values is a 2D ndarray (1, n_features)
    else:
        row = shap_values[0]

    return dict(zip(feat_names, row))


def diagnose_fields(bill, baseline, shap_values, pred_label):
    """
    Pinpoint which bill fields drove a flagged verdict using two layers:
      1. SHAP — measures each feature's contribution to the model decision
      2. Rules — hard checks that catch known billing error patterns

    Returns a list of findings ranked HIGH → MEDIUM → LOW.
    """
    findings = {}  # field -> best (highest-severity) finding for that field

    def add(field, bill_val, expected, deviation, severity, reason):
        rank = {"HIGH": 3, "MEDIUM": 2, "LOW": 1}
        if field not in findings or rank[severity] > rank[findings[field]["severity"]]:
            findings[field] = {
                "field":      field,
                "bill_value": round(bill_val,  2),
```

```
                "expected":   round(expected,  2),
                "deviation":  round(deviation, 2),
                "severity":   severity,
                "reason":     reason,
            }

    field_map = {
        "delta_total": ("total_amount",  bill["total_amount"],  baseline["total_amount"]),
        "delta_other": ("other_charges", bill["other_charges"], baseline["other_charges"]),
        "usage_kwh":   ("usage_kwh",     bill["usage_kwh"],     baseline["usage_kwh"]),
    }

    #SHAP
    contributions = get_shap_contributions(shap_values, pred_label)
    for feat, shap_val in contributions.items():
        if pred_label in ("Critical_Error", "Investigate") and shap_val > 0.05:
            field, bill_val, expected = field_map[feat]
            sev = "HIGH" if abs(shap_val) > 0.3 else "MEDIUM"
            add(field, bill_val, expected, bill_val - expected, sev,
                f"SHAP: '{feat}' pushed prediction by {shap_val:+.3f}")

    #rule-based checks
    total_dev = bill["total_amount"]  - baseline["total_amount"]
    other_dev = bill["other_charges"] - baseline["other_charges"]
    usage_dev = bill["usage_kwh"]     - baseline["usage_kwh"]

    #total inflated, other_charges unchanged → rate error / duplicate charge
    if total_dev > 15 and abs(other_dev) < 1.0:
        add("total_amount", bill["total_amount"], baseline["total_amount"], total_dev,
            "HIGH",
            f"Total is ${total_dev:+.2f} above customer average with no change in "
            f"other_charges — possible rate error or duplicate charge")

    #total and other_charges drop by the same amount → missing line item
    if total_dev < -8 and other_dev < -8 and abs(total_dev - other_dev) < 2.0:
        add("other_charges", bill["other_charges"], baseline["other_charges"], other_dev,
            "HIGH",
            f"Both total_amount and other_charges dropped ~${abs(other_dev):.2f} "
            f"together — likely a missing charge line item")

    #usage spike (z-score > 2.5 from customer average)
    usage_z = abs(usage_dev) / baseline["usage_std"]
    if usage_z > 2.5:
        add("usage_kwh", bill["usage_kwh"], baseline["usage_kwh"], usage_dev,
            "MEDIUM",
            f"Usage of {bill['usage_kwh']:.0f} kWh is {usage_z:.1f} std deviations "
            f"from customer average — verify meter reading")

    #sub-cent rounding noise (Benign, low severity)
    if 0 < abs(total_dev) < 0.05:
        add("total_amount", bill["total_amount"], baseline["total_amount"], total_dev,
            "LOW",
            f"Rounding discrepancy of ${total_dev:+.4f} — within tolerance")

    #HIGH first, then by absolute deviation descending
    ranked = sorted(findings.values(),
                    key=lambda x: ({"HIGH": 0, "MEDIUM": 1, "LOW": 2}[x["severity"]],
                                   -abs(x["deviation"])))
    return ranked


def print_diagnosis(bill, baseline, findings, pred_label, confidence, file_name):

    #Print per-bill diagnosis card
    sep = "—" * 60
    print(f"\n{sep}")
    print(f"FILE       : {file_name}")
    print(f"INVOICE    : {bill['invoice_number']}")
    print(f"VERDICT    : {pred_label}  ({confidence:.1f}% confidence)")
    print(f"BASELINE   : {baseline['source']}")
    print(sep)
    if not findings:
        print("  ✓  No specific field anomalies detected.")
        return
    print("  SUSPECTED FIELDS:")
    for i, f in enumerate(findings, 1):
        sign = "+" if f["deviation"] >= 0 else ""
        print(f"  [{i}] {f['severity']:<6}  {f['field']}")
        print(f"         Bill value : ${f['bill_value']:>10.2f}")
        print(f"         Expected   : ${f['expected']:>10.2f}")
        print(f"         Deviation  : {sign}${abs(f['deviation']):.2f}")
        print(f"         Reason     : {f['reason']}")
    print()
```

```
#inference loop
all_predictions = []

test_xml_dir   = "/content/content/test"
test_xml_files = sorted([f for f in os.listdir(test_xml_dir) if f.endswith(".xml")])

if not test_xml_files:
    print(f"No XML files found in {test_xml_dir}")
else:
    for file_name in test_xml_files:
        path = os.path.join(test_xml_dir, file_name)
        bill = parse_word_xml_bill(path)
        inv  = bill["invoice_number"]

        #dynamic per-customer baseline
        baseline = get_customer_baseline(inv)

        #build feature row using customer-aware deltas
        features_row = pd.DataFrame([{
            "delta_total": bill["total_amount"]  - baseline["total_amount"],
            "delta_other": bill["other_charges"] - baseline["other_charges"],
            "usage_kwh":   bill["usage_kwh"],
        }])


        pred_encoded = model.predict(features_row)[0]
        pred_probs   = model.predict_proba(features_row)[0]
        pred_label   = le.inverse_transform([pred_encoded])[0]
        confidence   = pred_probs.max() * 100

        #SHAP values for bill
        shap_vals = explainer.shap_values(features_row)

        #field-level diagnosis
        findings = diagnose_fields(bill, baseline, shap_vals, pred_label)

        #diagnosis card for flagged bills only
        if pred_label != "Benign":
            print_diagnosis(bill, baseline, findings, pred_label, confidence, file_name)

        #store res
        all_predictions.append({
            "invoice_number":   inv,
            "predicted_class":  pred_label,
            "confidence":       f"{confidence:.2f}%",
            "suspected_fields": ", ".join(f["field"] for f in findings) or "none",
            "top_severity":     findings[0]["severity"] if findings else "n/a",
            "top_reason":       findings[0]["reason"]   if findings else "n/a",
        })

        #update customer history so later bills in this batch benefit
        customer_history.setdefault(inv, []).append(bill)

predictions_df = pd.DataFrame(all_predictions)
print("\n" + "=" * 60)
print("SUMMARY TABLE")
print("=" * 60)
print(predictions_df[[
    "invoice_number", "predicted_class", "confidence",
    "suspected_fields", "top_severity"
]].to_string(index=False))
```

```
Extracted text preview:
<?xml version="1.0" encoding="UTF-8"?> <UtilityBill>     <CustomerInformation>         [cite_start]<Name>OXFORD LINCOLN [c

────────────────────────────────────────────────────────────
FILE       : 01_Oct26_Nov25.xml
INVOICE    : Number
VERDICT    : Critical_Error  (99.2% confidence)
BASELINE   : global_fallback
────────────────────────────────────────────────────────────
  SUSPECTED FIELDS:
  [1] HIGH    total_amount
         Bill value : $     44.81
         Expected   : $    599.03
         Deviation  : $554.22
         Reason     : SHAP: 'delta_total' pushed prediction by +3.701
  [2] MEDIUM  usage_kwh
         Bill value : $      0.00
         Expected   : $   1523.54
         Deviation  : $1523.54
```

```
        Reason     : Usage of 0 kWh is 3.9 std deviations from customer average — verify meter reading


Extracted text preview:
<?xml version="1.0" encoding="UTF-8"?> <UtilityBill>     <CustomerInformation>         [cite_start]<Name>OXFORD LINCOLN [c

────────────────────────────────────────────────────────────
FILE       : 02_Nov26_Dec27.xml
INVOICE    : Number
VERDICT    : Critical_Error  (99.0% confidence)
BASELINE   : customer_history (1 bills)
────────────────────────────────────────────────────────────
  SUSPECTED FIELDS:
  [1] HIGH    total_amount
        Bill value : $    111.60
        Expected   : $     44.81
        Deviation  : +$66.79
        Reason     : SHAP: 'delta_total' pushed prediction by +3.398


Extracted text preview:
<?xml version="1.0" encoding="UTF-8"?> <UtilityBill>     <CustomerInformation>         [cite_start]<Name>OXFORD LINCOLN [c

────────────────────────────────────────────────────────────
FILE       : 03_Dec28_Jan28.xml
INVOICE    : Number
VERDICT    : Critical_Error  (99.2% confidence)
BASELINE   : customer_history (2 bills)
────────────────────────────────────────────────────────────
  SUSPECTED FIELDS:
  [1] HIGH    total_amount
        Bill value : $      8.44
        Expected   : $     78.20
        Deviation  : $69.76
        Reason     : SHAP: 'delta_total' pushed prediction by +3.701
```

Adding Simulated Ground Truth Labels

```
predictions_df['ground_truth'] = ''

predictions_df.loc[[0, 2, 7, 11], 'ground_truth'] = 'Benign'
predictions_df.loc[[1, 3, 4, 5, 6, 8, 9, 10, 12], 'ground_truth'] = 'Critical_Error'

print("\nPredictions DataFrame with Ground Truth:")
print(predictions_df)
```

```
Predictions DataFrame with Ground Truth:
   invoice_number predicted_class confidence         suspected_fields  \
0          Number  Critical_Error     99.24%  total_amount, usage_kwh
1          Number  Critical_Error     98.98%             total_amount
2          Number  Critical_Error     99.24%             total_amount
3          Number          Benign     96.17%             total_amount
4          Number  Critical_Error     99.24%             total_amount
5          Number          Benign     96.17%                     none
6          Number  Critical_Error     99.24%             total_amount
7          Number  Critical_Error     99.24%             total_amount
8          Number  Critical_Error     99.24%             total_amount
9          Number  Critical_Error     99.24%             total_amount
10         Number  Critical_Error     99.24%             total_amount
11         Number  Critical_Error     99.24%             total_amount
12         Number  Critical_Error     98.98%             total_amount

   top_severity                                        top_reason  \
0          HIGH    SHAP: 'delta_total' pushed prediction by +3.701
1          HIGH    SHAP: 'delta_total' pushed prediction by +3.398
2          HIGH    SHAP: 'delta_total' pushed prediction by +3.701
3          HIGH  Total is $+17.49 above customer average with n...
4          HIGH    SHAP: 'delta_total' pushed prediction by +3.701
5           n/a                                               n/a
6          HIGH    SHAP: 'delta_total' pushed prediction by +3.701
7          HIGH    SHAP: 'delta_total' pushed prediction by +3.701
8          HIGH    SHAP: 'delta_total' pushed prediction by +3.701
9          HIGH    SHAP: 'delta_total' pushed prediction by +3.701
10         HIGH    SHAP: 'delta_total' pushed prediction by +3.701
11         HIGH    SHAP: 'delta_total' pushed prediction by +3.701
12         HIGH    SHAP: 'delta_total' pushed prediction by +3.398

      ground_truth
0           Benign
1   Critical_Error
2           Benign
3   Critical_Error
4   Critical_Error
```

```
5   Critical_Error
6   Critical_Error
7           Benign
8   Critical_Error
9   Critical_Error
10  Critical_Error
11          Benign
12  Critical_Error
```

```
from sklearn.metrics import classification_report

#extract predicted & ground truth labels
y_true = predictions_df['ground_truth']
y_pred = predictions_df['predicted_class']

#print the classification report
print("\nClassification Report for Test Bills:")
print(classification_report(y_true, y_pred))
```

```
Classification Report for Test Bills:
                precision    recall  f1-score   support

        Benign       0.00      0.00      0.00         4
Critical_Error       0.64      0.78      0.70         9

      accuracy                           0.54        13
     macro avg       0.32      0.39      0.35        13
  weighted avg       0.44      0.54      0.48        13
```